

Architecture Patterns for AI Applications

Module 8 — Business AI in Practice · Notebook 29

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

Contents

- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing
- 5 Course map

Outline

- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing
- 5 Course map

Architecture is the answer to a *traffic* question

Ask three things before you choose

- **How many users?** $1 \rightarrow 10 \rightarrow 100 \rightarrow 1,000+$.
- **How often, and who invokes it?** A human at a keyboard, a schedule, or another service?
- **How many teams** will own and ship pieces of it?

The point

Architecture is not about sophistication — it is about matching structure to load. The same feature wants a different shape at 5 users than at 5,000.

Outline

- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing
- 5 Course map

The four software patterns

Pattern	When it fits	Example
Single-tier	one analyst exploring, or one daily job	Jupyter notebook; cron + script
Three-tier	interactive use by non-developers	Streamlit + FastAPI + DB
Service-oriented	2–3 capabilities, one team	shared infra, common interface
Microservices	4+ teams shipping independently	one team per service

Read top to bottom as growing traffic and growing team count — not as “primitive to advanced”.

The over-engineering trap

Microservices for a 5-user pilot

Microservices became fashionable around 2015; many teams adopted them *before* they had the team size or traffic to justify the operational cost. The right unit of a service is a *capability* — not every Python module.

The rule

Pick the architecture you'll have *outgrown in 12 months*, not the one you'll *need in 36*. Migrations are real, but rarely the thing that kills a project — premature complexity is.

Outline

- 1 Traffic
- 2 Patterns
- 3 Pipeline**
- 4 Choosing
- 5 Course map

The cross-cutting ML / AI pipeline

What pure-software patterns don't capture

- **Training data is part of the artefact.** Two models that look identical but trained on different snapshots are *different products*.
- **Drift exists.** 92 % accurate in March is not 92 % in October — monitoring must compare against fresh ground truth.
- **The feedback loop has economic cost.** Every prediction nudges the world the next training batch will learn from.

Data → features → train → evaluate → serve → *monitor* → (back to data).

Outline

- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing**
- 5 Course map

Choosing the right size

Smells that you've outgrown your current shape

- Someone asks “can we run this nightly?” → add scheduling (single-tier job).
- A non-developer needs to use it → add a UI tier (three-tier).
- A second team wants to build on it → expose it as a service.

Two opposite mistakes to avoid

Over-engineering (microservices for a pilot) and *under-engineering* (a 200-user system held together by one notebook).

Outline

- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing
- 5 Course map

How the course maps to these patterns

Pattern	Where it appears in the course
Single-tier (script / notebook)	Every notebook NB 1–27, by default
Scheduled single-tier	NB 24 — scheduling & orchestration
Three-tier with AI in the middle	NB 34 (POC 2) and NB 27 capstone
Service-oriented	<code>llm_providers.py</code> — pluggable providers
Microservices	<i>patterns only</i> — too big for a teaching course
ML pipeline	NB 15–17 + NB 24 — train / evaluate / serve

You *build* the three-tier and ML-pipeline shapes in **Notebook 34**; the rest you meet as patterns here.